

Chapter 11: Classes

By Victoria Gonda

In this chapter, you'll get acquainted with **classes**, which are **named types**. Classes are one of the cornerstones of **object-oriented programming**, a style of programming where the types have both data and behavior. In classes, data takes the form of **properties** and behavior is implemented using functions called **methods**.

Open the chapter starter project to start learning!

Creating classes

Consider the following class definition in Kotlin and add it to your Kotlin file, outside of any `main()` function:

```
class Person(var firstName: String, var lastName: String) {  
    val fullName  
    get() = "$firstName $lastName"  
}
```

That's simple enough! The keyword `class` is followed by the name of the class, `Person`. Inside the parentheses after the class name is the **primary constructor** for the class, and for `Person` you're indicating that there are two mutable `String` properties, `firstName` and `lastName`. You'll see how to create other constructors in Chapter 15, "Advanced Classes." Everything in the curly braces is a member of the class.

Note: You can place classes in a file alongside other constructs, or in their own file. For this exercise, you'll put everything in the same file.

You create an **instance** of a class by using the class name and passing in arguments to the constructor. Add this to your `main()` function:

```
val john = Person(firstName = "Johnny", lastName = "Appleseed")
```

The class instances are the **objects** of object-oriented programming, not to be confused with the Kotlin object keyword.

`Person` has another property named `fullName` with a **custom getter** that uses the other properties in its definition:

```
println(john.fullName) // > Johnny Appleseed
```

It uses both the `firstName` and `lastName` properties to compute the `fullName`, "Johnny Appleseed".

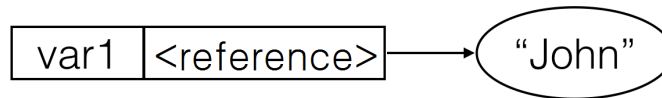
Reference types

In Kotlin, an instance of a class is a mutable object. Classes are **reference types**. This means a variable of a class type does not store an actual instance, but a **reference** to a location in memory that stores the instance.

Create a SimplePerson class and instance with only a name like this:

```
class SimplePerson(val name: String)
var var1 = SimplePerson(name = "John")
```

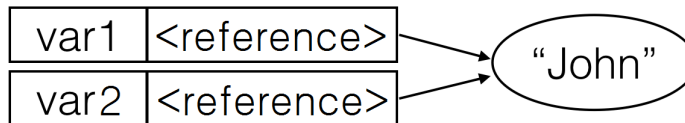
It looks something like this in memory:



Now, create a new variable var2 and assign to it the value of var1:

```
var var2 = var1
```

The references inside both var1 and var2 reference the same place in memory:



The heap vs. the stack

When you create a reference type such as a class, the system stores the actual instance in a region of memory known as the **heap**. References to the class instances are stored in a region of memory called the **stack**, unless the reference is part of a class instance, in which case the reference is stored on the heap with the rest of the class instance.

Both the heap and the stack have essential roles in the execution of any program:

- The system uses the **stack** to store anything on the immediate thread of execution; it is tightly managed and optimized by the CPU. When a function creates a variable, the stack stores that variable and then destroys it when the function exits. Since the stack is so well organized, it's very efficient, and thus quite fast.
- The system uses the **heap** to store instances of reference types. The heap is generally a large pool of memory from which the system can request and dynamically allocate blocks of memory. The lifetime is flexible and dynamic. The heap doesn't automatically destroy its data like the stack does; additional work is required to do that. This makes creating and removing data on the heap a slower process, compared to on the stack.

When you create an instance of a class, your code requests a block of memory on the heap to store the instance itself. It stores the *address* of that memory in your named variable on the stack.

This has only been a brief introduction to the dynamics of heaps and stacks, but you know enough at this point to understand the reference semantics you'll use to work with classes.

Working with references

Since a class is a reference type, when you assign to a variable of a class type, the system does *not* copy the instance; only a reference is copied.

Add the following code to the bottom of your `main()` function:

```
var homeOwner = john
john.firstName = "John"

println(john.firstName)      // > John
println(homeOwner.firstName) // > John
```

Here, you assign a new variable, `homeOwner`, to the `john` object.

Run the code to see the results. Even though you only changed the first name for `john`, the name was also changed for `homeOwner`, since they both reference the same object. As you can see, `john` and `homeOwner` truly have the same data!

This implied sharing among class instances results in a new way of thinking when passing things around. For instance, if the `john` object changes, then anything holding a reference to `john` will automatically see the update.

Mini-exercise

Change the value of `lastName` on `homeOwner`, then try reading `fullName` on both `john` and `homeOwner`. What do you observe?

Object identity

In the previous code sample, it's easy to see that `john` and `homeOwner` are pointing to the same object. The code is short and both references are named variables. What if you want to see if the value behind a variable *is* John?

You might think to check the value of `firstName`, but how would you know it's the John you're looking for and not an impostor? Or worse, what if John changed his name again?

In Kotlin, the `===` operator lets you check if the *identity* of one object is equal to the identity of another. Add the following line of code to your example:

```
println(homeOwner === john) // > true
```

Run and observe the results. Similar to how the `==` operator checks if two *values* are equal, the `===` identity operator compares the memory address of two *references*. It tells you whether the value of the references are the same; that is, they point to the same block of data on the heap.

Add the following code. You can add print statements if you want to see the results for yourself:

```
val impostorJohn = Person(firstName = "John", lastName =  
    "Appleseed")  
  
john === homeOwner // true  
john === impostorJohn // false  
impostorJohn === homeOwner // false  
  
// Assignment of existing variables changes the instances the  
// variables reference.  
homeOwner = impostorJohn  
john === homeOwner // false  
  
homeOwner = john  
john === homeOwner // true
```

This shows how the `===` operator can tell the difference between the John you're looking for and an imposter-John.

This can be particularly useful when you cannot rely on regular equality (==) to compare and identify objects you care about. Try the following code:

```
// Create fake, imposter Johns.
var imposters = (0..100).map {
    Person(firstName = "John", lastName = "Appleseed")
}

// Equality (==) is not effective when John cannot be identified
// by his name alone
imposters.map {
    it.firstName == "John" && it.lastName == "Appleseed"
}.contains(true) // true
```

In the above code, you can see how the equality operator is not sufficient to identify the original John.

Now, try this code:

```
// Check to ensure the real John is not found among the
// imposters.
println(imposters.contains(john)) // > false

// Now hide the "real" John somewhere among the imposters.
val mutableImposters = mutableListOf<Person>()
mutableImposters.addAll(imposters)
mutableImposters.contains(john) // false
mutableImposters.add(Random().nextInt(5), john)

// John can now be found among the imposters.
println(mutableImposters.contains(john)) // > true

// Since `Person` is a reference type, you can use === to grab
// the real John out of the list of imposters and modify the value.
// The original `john` variable will print the new last name!
val indexOfJohn = mutableImposters.indexOf(john)
if (indexOfJohn != -1) {
    mutableImposters[indexOfJohn].lastName = "Bananapeel"
}

println(john.fullName) // > John Bananapeel
```

Note: You have to import the `java.util.*` package in order to work with the `Random()` class.

By using the identity operator, you can verify that the *references* themselves are equal, and separate your real John from the crowd.

You may actually find that you won't use the identity operator `===` very much in your day-to-day Kotlin. What's important is to understand what it does, and what it demonstrates about the properties of reference types.

Mini-exercise

Write a function `memberOf(person: Person, group: List<Person>): Bool` that will return `true` if `person` can be found inside `group`, and `false` if it can not.

Test it by creating two arrays of five `Person` objects for `group` and using `john` as the person. Put `john` in one of the arrays, but not in the other.

Methods and mutability

As you've read before, instances of classes are mutable objects. Create the classes `Student` and `Grade` as defined below:

```
class Grade(  
    val letter: String,  
    val points: Double,  
    val credits: Double  
)  
  
class Student(  
    val firstName: String,  
    val lastName: String,  
    val grades: MutableList<Grade> = mutableListOf(),  
    var credits: Double = 0.0  
) {  
  
    fun recordGrade(grade: Grade) {  
        grades.add(grade)  
        credits += grade.credits  
    }  
}
```

Now, use these classes in your `main()` function:

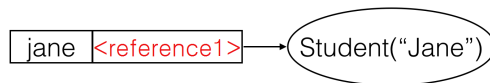
```
val jane = Student(firstName = "Jane", lastName = "Appleseed")  
val history = Grade(letter = "B", points = 9.0, credits = 3.0)  
var math = Grade(letter = "A", points = 16.0, credits = 4.0)  
  
jane.recordGrade(history)  
jane.recordGrade(math)
```

Note that `recordGrade()` can mutate the array `grades` by adding more values to the end. Like any mutable list, `grades` can be added to even though the `grades` reference itself is immutable. This is independent of the fact that `jane` is marked as an immutable `val` reference. Similarly, the `credits` `Double` value can be changed in `recordGrade()` because it's defined as a mutable `var` within the `Student` class.

Mutability and constants

The previous example may have had you wondering how you were able to modify `jane` even though it was defined as a constant `val`.

When you define a constant, the value of the constant cannot be changed. It is important to remember that, with reference types, the value is a *reference*.



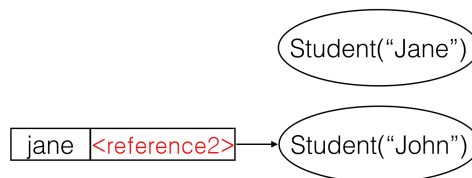
The value of “reference1” in red is the value stored in `jane`. This value is a reference and because `jane` is declared as a constant, this reference is constant. If you were to attempt to assign another student to `jane`, you would get a build error:

```
// Error: jane is a `val` constant
jane = Student(firstName = "John", lastName = "Appleseed")
```

If you declared `jane` as a variable instead, you would be able to assign to it another instance of `Student` on the heap:

```
var jane = Student(firstName = "Jane", lastName = "Appleseed")
jane = Student(firstName = "John", lastName = "Appleseed")
```

After the assignment of another `Student` to `jane`, the reference value behind `jane` would be updated to point to the new `Student` object.



Since nothing would be referencing the original “Jane” object, its memory would be freed to use elsewhere.

Any individual member of a class can be protected from modification through the use of constants, but because reference types are not *themselves* treated as values, they are not protected as a whole from mutation, even when instantiated with `val`.

Mini-exercise

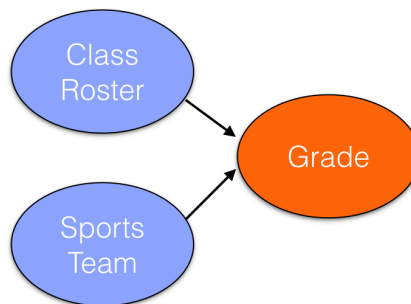
Add a property with a custom getter to `Student` that returns the student's Grade Point Average, or GPA. A GPA is defined as the number of points earned divided by the number of credits taken. For the example above, Jane earned $(9 + 16 = 25)$ points while taking $(3 + 4 = 7)$ credits, making her GPA $(25 / 7 = 3.57)$.

Note: Points in most American universities range from 4 per credit for an A, down to 1 point for a D (with an F being 0 points). For this exercise, you may of course use any scale that you want!

Understanding state and side effects

The referenced and mutable nature of classes leads to numerous programming possibilities, as well as many concerns. If you update a class instance with a new value, then every reference to that instance will also see the new value.

You can use this to your advantage. Perhaps you pass a `Student` instance to a sports team, a report card and a class roster. Imagine all of these entities need to know the student's grades, and because they all point to the same instance, they'll all see new grades as the instance records them.



The result of this sharing is that class instances have **state**. Changes in state can sometimes be obvious, but often they're not.

To illustrate this, consider the `credits` property of the `Student` class, which is initialized as:

```
var credits = 0.0
```

The `recordGrade()` member mutates this `credits` property:

```
fun recordGrade(grade: Grade) {  
    grades.add(grade)  
    credits += grade.credits  
}
```

Calling `recordGrade()` has the side effect of updating `credits`.

Now, observe how side effects can result in non-obvious behavior by adding this code:

```
println(jane.credits) // 7  
  
// The teacher made a mistake; math has 5 credits  
math = Grade(letter = "A", points = 20.0, credits = 5.0)  
jane.recordGrade(math)  
  
println(jane.credits) // 12, not 8!
```

Run your new code. Whoever wrote the `Student` class did so somewhat naïvely by assuming that the same grade won't get recorded twice! But, because `math` was already added in a previous example, it was recorded twice. Because class instances are mutable, you need to be careful about unexpected behavior around shared references.

While confusing in a small example such as this, mutability and state could be extremely jarring as classes grow in size and complexity. Situations like this would be much more common with a `Student` class that scales to 20 properties and has 10 methods.

Data classes

Suppose you want to define a `Student` class and have added functionality, such as the ability to compare whether two students are equal in value or the ability to easily print the student data. You might define the class as follows:

```
class Student(  
    var firstName: String,  
    var lastName: String,  
    var id: Int  
) {  
  
    override fun hashCode(): Int {  
        val prime = 31  
        var result = 1  
  
        result = prime * result + firstName.hashCode()  
        result = prime * result + id  
        result = prime * result + lastName.hashCode()  
  
        return result  
    }  
  
    override fun equals(other: Any?): Boolean {  
        if (this === other)  
            return true  
  
        if (other == null)  
            return false  
  
        if (javaClass != other.javaClass)  
            return false  
  
        val obj = other as Student?  
  
        if (firstName != obj?.firstName)  
            return false  
  
        if (id != obj.id)  
            return false  
  
        if (lastName != obj.lastName)  
            return false  
  
        return true  
    }  
  
    override fun toString(): String {  
        return "Student (firstName=$firstName, lastName=$lastName,  
id=$id)"  
    }  
  
    fun copy(  
        firstName: String = this.firstName,  
        lastName: String = this.lastName,  
        id: Int = this.id  
    ) = Student(firstName, lastName, id)  
}
```

You've also added a hash code for each student, and a function to copy one student into another.

Classes with a primary purpose for holding data are very common in programming. They are especially used as **model** objects in many programming patterns that attempt to model real world objects.

When using these model classes, comparing instances, printing them and copying them are all very common actions:

```
val albert = Student(
    firstName = "Albert",
    lastName = "Einstein",
    id = 1
)
val richard = Student(
    firstName = "Richard",
    lastName = "Feynman",
    id = 2
)
val albertCopy = albert.copy()

println(albert)
// > Student (firstName=Albert, lastName=Einstein, id=1)
println(richard)
// > Student (firstName=Richard, lastName=Feynman, id=2)
println(albert == richard) // > false
println(albert == albertCopy) // > true
println(albert === albertCopy) // > false
```

Using the == operator with the instances compares the values in the objects using the equals() function, whereas === compares the identity of the references, as was discussed above.

These actions on instances are so common that Kotlin provides a variation on classes named **data classes**. By using data classes, you can avoid having to declare all the boilerplate code that was used in our re-definition of Student.

You define a data class just like a regular class except that you prepend the class keyword with data:

```
data class StudentData(
    var firstName: String,
    var lastName: String,
    var id: Int
)
```

Check out the data class in action. Add the following:

```
val marie = StudentData("Marie", "Curie", id = 1)
val emmy = StudentData("Emmy", "Noether", id = 2)
val marieCopy = marie.copy()

println(marie)
// > StudentData(firstName=Marie, lastName=Curie, id=1)
println(emmy)
// > StudentData(firstName=Emmy, lastName=Noether, id=2)
println(marie == emmy) // > false
println(marie == marieCopy) // > true
println(marie === marieCopy) // > false
```

Now, run it to see the results. The `StudentData` data class has all the same functionality as the new `Student` class, and it's all defined in one statement!

Destructuring declarations

You can extract the data inside of a data class using a **destructuring declaration**. Just assign a variable to each of the properties of the data class in one assignment statement:

```
val (firstName, lastName, id) = marie

println(firstName) // > Marie
println(lastName)  // > Curie
println(id)        // > 1
```

Destructuring declarations are particularly useful in returning more than one value from a function. They also work in other contexts, for example, in for loops over map objects.

Be careful, as the variable you're assigning the properties to do not need to have the same name as the property. You could accidentally swap the first and last names! The properties are destructured in the same order as in the constructor.

Challenges

Here are some challenges for you to practice your new knowledge. If you get stuck at any point, check out the solutions in the materials for this chapter.

Challenge 1: Movie lists

Imagine you're writing a movie-viewing application in Kotlin. Users can create lists of movies and share those lists with other users.

Create a `User` class and a `MovieList` class that maintains lists for users.

- `MovieList`: Contains a name and a mutable list of movie titles. The name and titles can all be represented by `Strings`. A `print` method will print all the movies in the movie list.
- `User`: Has a method `addList()` which adds the given `MovieList` to a mutable map of `MovieList` objects (using the name as a key), and `list(name: String): MovieList?` which will return the `MovieList` for the provided name.
- Create `jane` and `john` users and have them create and share lists. Have both `jane` and `john` modify the same list and call `print` from both users. Are all the changes reflected?

Challenge 2: T-Shirt store — data classes

Your challenge here is to build a set of objects to support a T-shirt store. Decide if each object should be a class or a data class, and go ahead and implement them all.

- `TShirt`: Represents a shirt style you can buy. Each `TShirt` has a size, color, price, and an optional image on the front.
- `User`: A registered user of the t-shirt store app. A user has a name, email, and a `ShoppingCart` (see below).
- `Address`: Represents a shipping address, containing the name, street, city, and zip code.
- `ShoppingCart`: Holds a current order, which is composed of a list of `TShirts` that the `User` wants to buy, as well as a method to calculate the total cost. Additionally, there is an `Address` that represents where the order will be shipped.

Key points

- **Classes** are a named type that can have properties and methods.
- Classes use **references** that are shared on assignment.
- Class instances are called **objects**.
- Objects are **mutable**.
- Mutability introduces **state**, which adds complexity when managing your objects.
- **Data classes** allow you to create simple model objects that avoid a lot of boilerplate for comparing, printing, and copying objects.
- **Destructuring declarations** allow you to easily extract multiple properties of data class objects.

Where to go from here?

You've just scratched the surface of the power and usage of classes!

In the next few chapters, you'll learn more details about class properties and methods as well as advanced usage of classes including inheritance. You'll also take a look at the `object` keyword, which is used when you want to ensure that only one instance of a type is created in your application.